Nicholas Alan Gilbert

Empircal Exercise # 1

For Completion of FIN 9777: Data Science in Finance at Baruch College MS Finance

Navigation

Set Up

Import

- Reflect Task 1

Daily Returns

- Reflect Task 2

Monthly Returns

- Reflect Task 3

Setup

Back to Top

Import necessary libraries:

`import yfinance` for financial data

`import pandas as pd` for data manipulation and handling; `pd` is that standard alias for this library

`import matplotlib.pyplot as plt` for data visualization

In [1]:
```
import yfinance
import pandas as pd
import matplotlib.pyplot as plt
import scipy.stats as st
```

```
import numpy as np
```

Here we will define custom 'helper functions' used throughout the project

In [2]:
```
def summary_statistics_table(dataframe):
    """
    This helper-function takes a DataFrame of Daily Returns
    with columns of Ticker-names, and rows of dates.
    Returns a DataFrame with summary statistics:
    'Mean', 'Median', 'Standard-Deviation', 'Skewness', 'Kurtosis', 'M
    utilizing pandas built-in .mean(), .median(), ..., etc. methods

    """

    columns = dataframe.columns

    index_list = ['Mean', 'Median', 'Standard-Deviation', 'Skewness',

    member_stats = pd.DataFrame(data=None, index=index_list)
    for col in columns:
        rtn:pd.Series = dataframe[col]

        mean = rtn.mean()
        median = rtn.median()
        std = rtn.std()
        skew = rtn.skew()
        kurt = rtn.kurt()
        min_rtn = rtn.min()
        max_rtn = rtn.max()



        stats = pd.Series(
            name= col,
            data = [
                mean,
                median,
                std,
                skew,
                kurt,
                min_rtn,
                max_rtn
            ],
            index = index_list,
        )
        member_stats = member_stats.join(stats)

    return member_stats

def return_histograms(dataframe: pd.DataFrame):
```

```
    """
    This will display a series of histograms with summary statistics l
    It accepts a pandas DataFrame with Returns data. Row Index of date
    Columns of different tickers.
    It Depends on our helper-function summary_statistics_table()
    """

    stats = summary_statistics_table(dataframe)

    columns = dataframe.columns

    for col in columns:
        plt.grid()
        counts,bins,_ = plt.hist(
            dataframe[col],
            bins = 100,
            color = 'deeppink',
            edgecolor = 'black',
            linewidth = 0.2,
            alpha=0.3
        )

        max_occ = counts.max()

        #Plot our mean on distribution
        mean = stats[col]['Mean']
        plt.vlines(
            mean,
            ymin=0,
            ymax= 0.95* max_occ,
            color='purple',
            label=f"Mean={mean:.4f}")
        #Plot +/- standard-deviation
        std = stats[col]['Standard-Deviation']
        std_up = mean + std
        std_down = mean - std
        plt.vlines(
            [std_up, std_down],
            ymin=0,
            ymax= 0.4* max_occ,
            color='blue',
            alpha=0.7,
            label=rf"$\sigma = \pm${std:.4f}")
        #Plot 2-sigma moves
        std_up2 = mean + std*2
        std_down2 = mean - std*2
        plt.vlines(
            [std_up2, std_down2],
            ymin=0,
            ymax= 0.2* max_occ,
            color='blueviolet',
            alpha=0.7,
```

```
            label=rf"2$\sigma = \pm${std*2:.4f}")
        #Plot 3-sigma moves
        std_up2 = mean + std*3
        std_down2 = mean - std*3
        plt.vlines(
            [std_up2, std_down2],
            ymin=0,
            ymax= 0.1* max_occ,
            color='red',
            alpha=0.7,
            label=rf"3$\sigma =  \pm${std*3:.4f}")

        #Overlay a Bell-Curve

        xmin, xmax = plt.xlim()
        x = np.linspace(xmin, xmax, 100)

        p = st.norm.pdf(x, mean, std)



        bin_width = bins[1] - bins[0]
        scaled_p = p * len(dataframe[col].dropna()) * bin_width
        plt.plot(x,scaled_p, color='black', alpha=0.3, label="Normal-D

        #Clean Up plot and label
        plt.ylabel("# of occurrences")
        plt.xlabel("Return in %-decimal: ie. 1% = 0.01")
        plt.title(f"{col} Return Distribution")
        plt.legend()

        plt.show()
```

Import

Back to Top

Let's instantiate a list of ticker symbols for stocks we would like to perform analysis on

---

AAPL - Apple Inc - Technology

TRMD - Torm LLC. - Industrial Shipping

LMT - Lockheed Marting - Aerospace and Defense

TGT - Target Corp - Consumer Retail

DX - Dynex Capital - Real-Estate, mREIT

SPY - SPDR S&P 500 ETF Trust - Benchmark Index

We pull in historical data utilizing `yfinance` library. yfinance is referred to as an "API", short for Application-Programatic Interface. yfinance is actually a script that is a glorified webscraper. It is simple and straighforward to use; but one should not depend on it for robust financial applications. For example if you start making high-frequency calls Yahoos server's will restrict your IP address. If Yahoo changes the layout of their webpages then yfinance might break. Nontheless, yfinance is straight-forward and simple to use. It is good-enough for small learning exercises; but it should be noted that it should not be depended on for production use cases.

In [3]:
```
# Let's instantiate a list of ticker symbols for stocks we would like

# AAPL - Apple Inc - Technology
# TRMD - Torm LLC. - Industrial Shipping
# LMT - Lockheed Marting - Aerospace and Defense
# TGT - Target Corp - Consumer Retail
# DX - Dynex Capital - Real-Estate, mREIT
# SPY - SPDR S&P 500 ETF Trust - Benchmark Index

tickers = ['AAPL', 'TRMD', 'LMT', 'TGT', 'DX', 'SPY']

# Now we can grab data on these stocks by calling yfinance methods

yf_tickers = yfinance.Tickers(tickers)

# We call the `.history()` method on our yfinance Tickers object to ca
# We can pass parameters to the function to define our date range

data = yf_tickers.download(start='1995-01-01', end='2026-08-31')
```

```
[**********************100%**********************]  6 of 6 completed
```

In [4]:
```
data
```

Out[4]:

Price	Capital Gains					
Ticker	SPY	AAPL	DX	LMT	SPY	TGT
Date						
1995-01-03	0.0	0.283707	5.055719	10.161205	26.378325	3.117848
1995-01-04	0.0	0.291100	4.938149	10.213856	26.504368	3.112277
1995-01-05	0.0	0.287403	5.526023	10.354253	26.504368	3.162387
1995-01-06	0.0	0.310506	5.643597	10.441998	26.531363	3.140116
1995-01-09	0.0	0.304615	5.584808	10.441998	26.558388	3.051035
...	...	...	...	...	...	...
2026-08-24	0.0	310.339996	12.867857	560.672119	761.578796	169.889999	2
2026-08-25	0.0	309.899994	12.828416	553.098999	764.012756	163.470001	2
2026-08-26	0.0	313.450012	12.700230	562.143005	764.182373	164.039993	2
2026-08-27	0.0	314.579987	12.719950	562.411377	769.189941	165.929993	2
2026-08-28	0.0	319.700012	12.729811	560.383850	767.444275	163.179993	3

7967 rows × 43 columns

In [5]:
```
# We are only concerned with daily returns at this point. We can calcu
price_history = data[['Close', 'Volume']]
price_history
```

Out[5]:

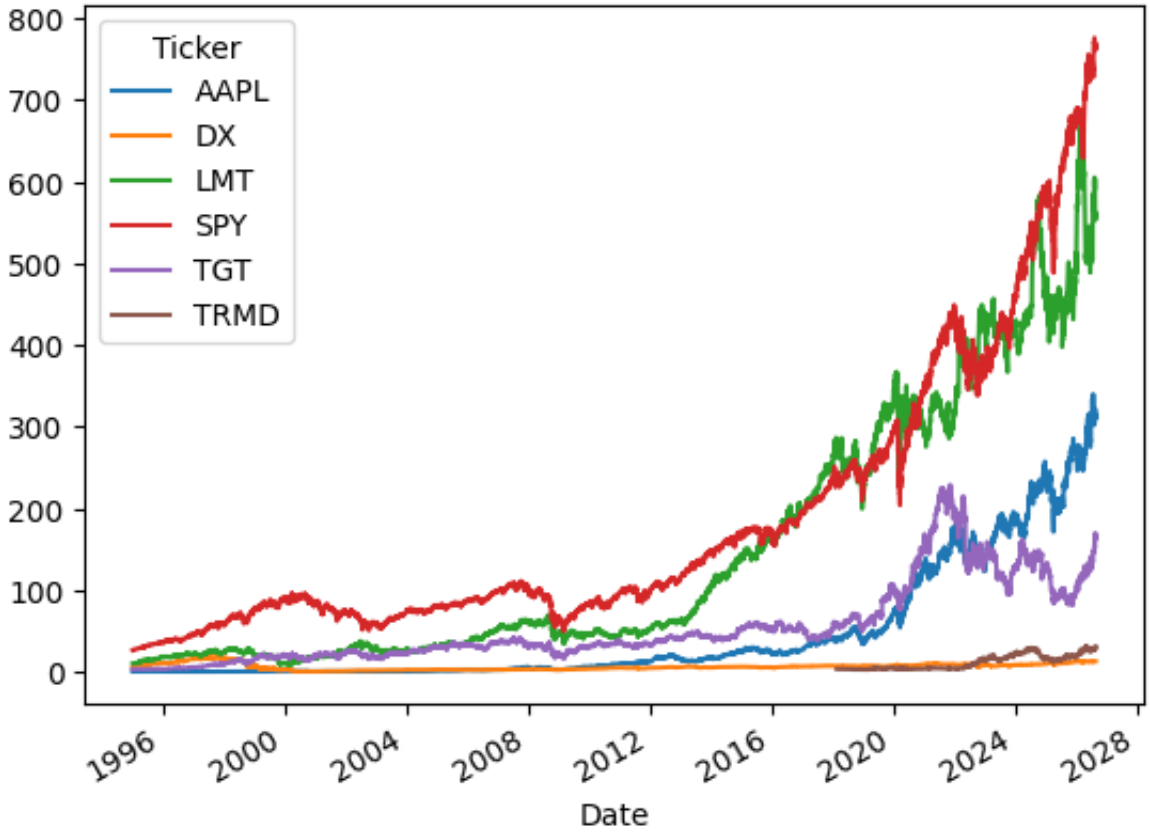
Price						Close
Ticker	AAPL	DX	LMT	SPY	TGT	TRMD
Date						
1995-01-03	0.283707	5.055719	10.161205	26.378325	3.117848	NaN
1995-01-04	0.291100	4.938149	10.213856	26.504368	3.112277	NaN
1995-01-05	0.287403	5.526023	10.354253	26.504368	3.162387	NaN
1995-01-06	0.310506	5.643597	10.441998	26.531363	3.140116	NaN
1995-01-09	0.304615	5.584808	10.441998	26.558388	3.051035	NaN
...	...	...	...	...	...	...
2026-08-24	310.339996	12.867857	560.672119	761.578796	169.889999	29.466675
2026-08-25	309.899994	12.828416	553.098999	764.012756	163.470001	29.308352
2026-08-26	313.450012	12.700230	562.143005	764.182373	164.039993	28.013830
2026-08-27	314.579987	12.719950	562.411377	769.189941	165.929993	29.615685
2026-08-28	319.700012	12.729811	560.383850	767.444275	163.179993	30.379358

7967 rows × 12 columns

In [6]:
```
# This is now a Panda's DataFrame object that will be easy to work wit
# Let's take a look at a simple plot of the data with the .plot() meth

price_history['Close'].plot()
```

Out[6]:
```
<Axes: xlabel='Date'>
```



Reflect Task 1

Back to Top

Right out of the box we can view the price history of our stocks pretty easily. This is a bit of an "apples to oranges" comparison. The fact that one stock's price per share differs from another doesn't tell us much about the performance of that company over time.

We want to make this 'apples to apples', or a common-size comparison of price performance over time. We do that by looking calculated returns.

Daily Returns

Back to Top

Next we want to compute daily returns.

We usually use log-returns for heavy-duty quantitative work, with an exploratory analysis such as this we will use simple period returns. Defined below.

Where;

$ r_t = $ return at time period t

$ p_t = $ asset price at time t

$ t = $ time-period eg. daily, weekly, monthly

Asset Return:

$$ r_t = \frac{p_t}{p_{t-1}} - 1$$

Equivalently:

$$ r_t = \frac{p_t - p_{t-1} }{p_{t-1}} $$

I prefer the first expression and I will proof that they are equal:

$$ \frac{p_t - p_{t-1} }{p_{t-1}} = \frac{p_t}{p_{t-1}} - \frac{p_{t-1}}{p_{t-1}} $$ $$ \frac{p_t - p_{t-1} }{p_{t-1}} = \frac{p_t}{p_{t-1}} - 1 $$

Thus our calculation for daily returns:

$$ \frac{r_{\text{today}}} {r_{\text{yesterday}}} - 1 $$

pandas has a method that will do this for us. Let's impliment ourselves and check against pandas' built-in. These little checks are called 'sanity checks' and good proof to the developer that they understand what is going on within their model.

In [7]:
```
today = price_history['Close']
yesterday = price_history['Close'].shift(1) # This 'shifts' entire col

returns = today / yesterday - 1
pandas_return = price_history['Close'].pct_change(fill_method=None)
```

In [8]:
```
#Sanity Check

sanity_check = pandas_return.dropna() == returns.dropna()

sanity_check
```

Out[8]:

Ticker	AAPL	DX	LMT	SPY	TGT	TRMD
Date						
2018-02-26	True	True	True	True	True	True
2018-02-27	True	True	True	True	True	True
2018-02-28	True	True	True	True	True	True
2018-03-01	True	True	True	True	True	True
2018-03-02	True	True	True	True	True	True
...	...	...	...	...	...	...
2026-08-24	True	True	True	True	True	True
2026-08-25	True	True	True	True	True	True
2026-08-26	True	True	True	True	True	True
2026-08-27	True	True	True	True	True	True
2026-08-28	True	True	True	True	True	True

2139 rows × 6 columns

In [9]:
```
sanity_check.describe()
```

Out[9]:

Ticker	AAPL	DX	LMT	SPY	TGT	TRMD
count	2139	2139	2139	2139	2139	2139
unique	1	1	1	1	1	1
top	True	True	True	True	True	True
freq	2139	2139	2139	2139	2139	2139

Sanity Check Worked Out

`.dropna()` ignores missing data that is labelled NaN (Not a Number)

Doing a simple comparison between our calculated values and `pandas` built-in `.pct_change()` method is all `True` values. So we will go ahead and utlize `pandas .pct_change()` to calculate simple returns from here on out.

These little sanity checks are crucial for proving that we understand what is happening and helps us avoid mistakes that could invalidate our analysis downstream.

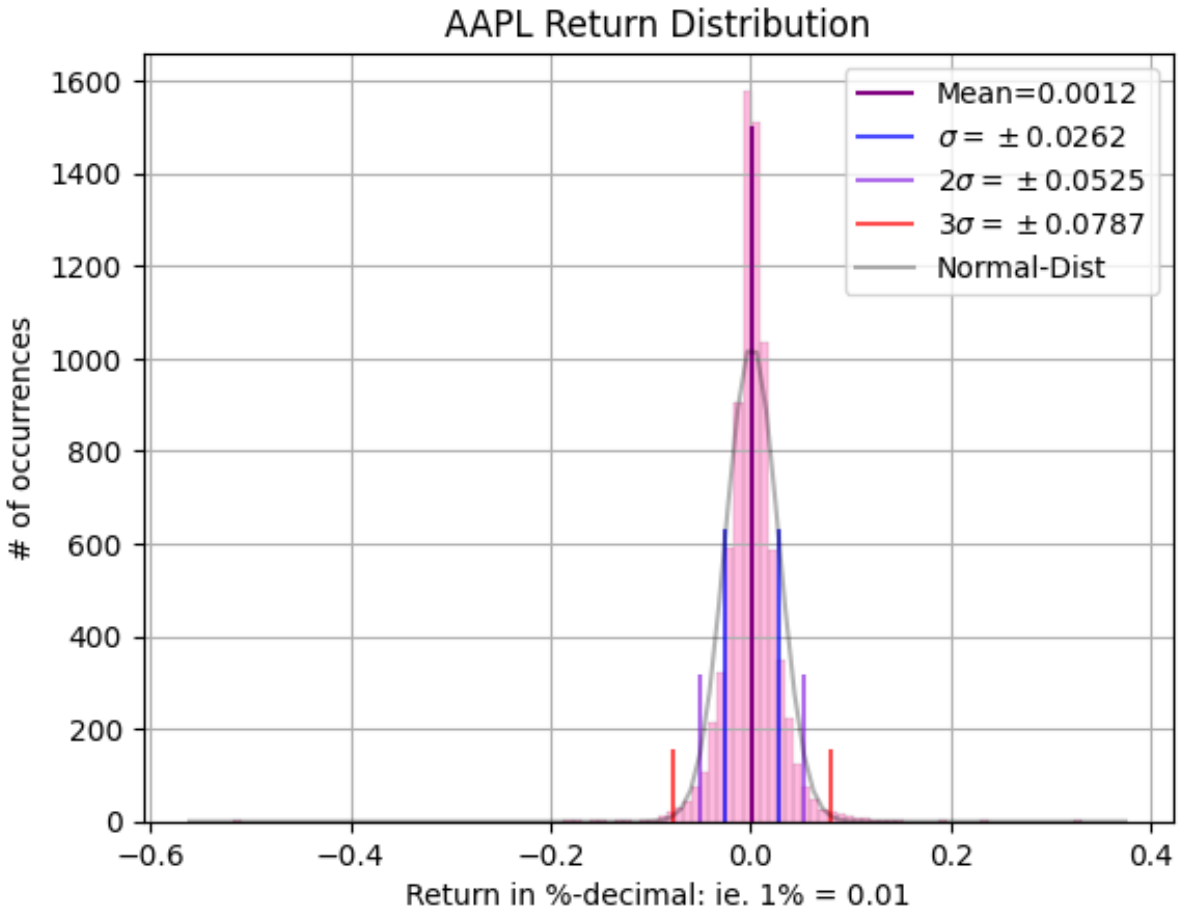
In [10]:
```
#We will call our helper-function to output Summary Stats
daily_returns = price_history['Close'].pct_change(fill_method=None)
summary_daily = summary_statistics_table(daily_returns)

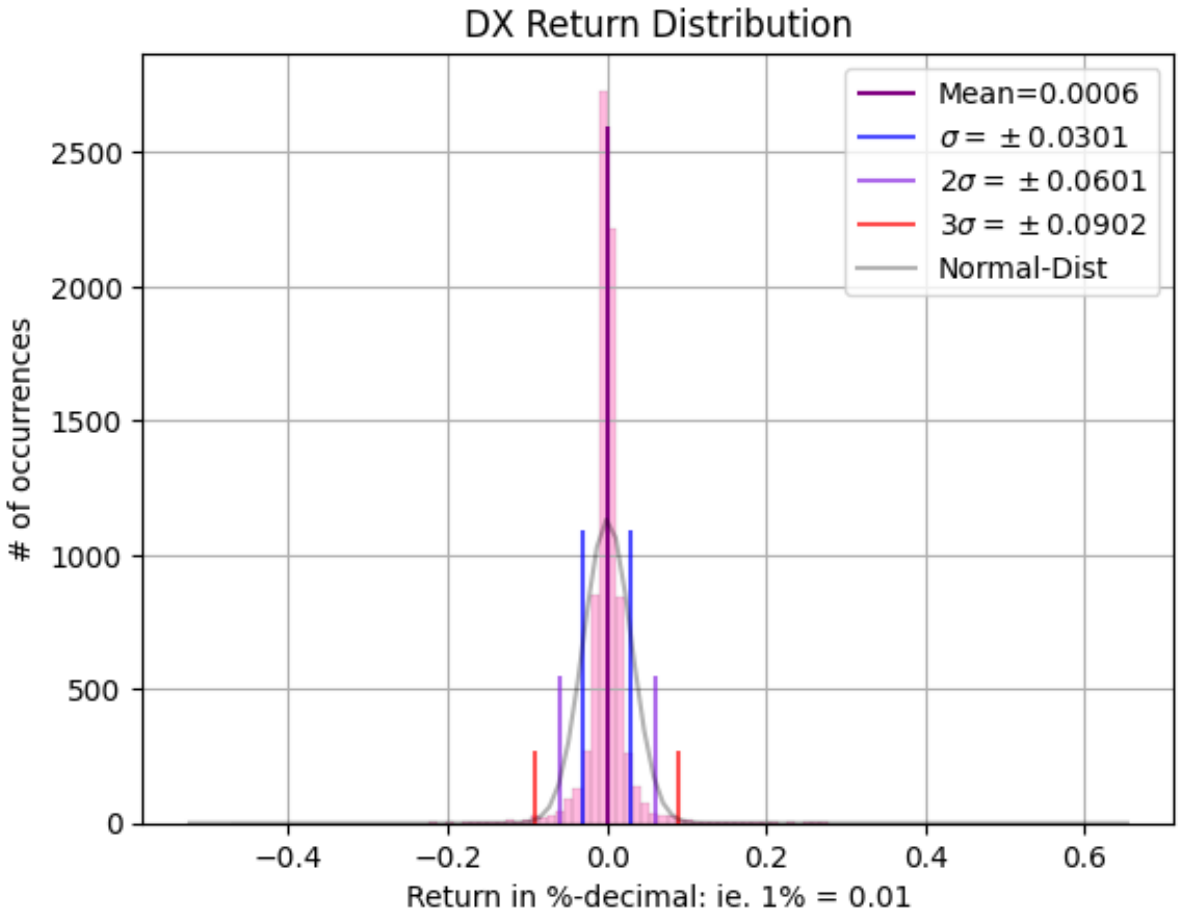

summary_daily
```

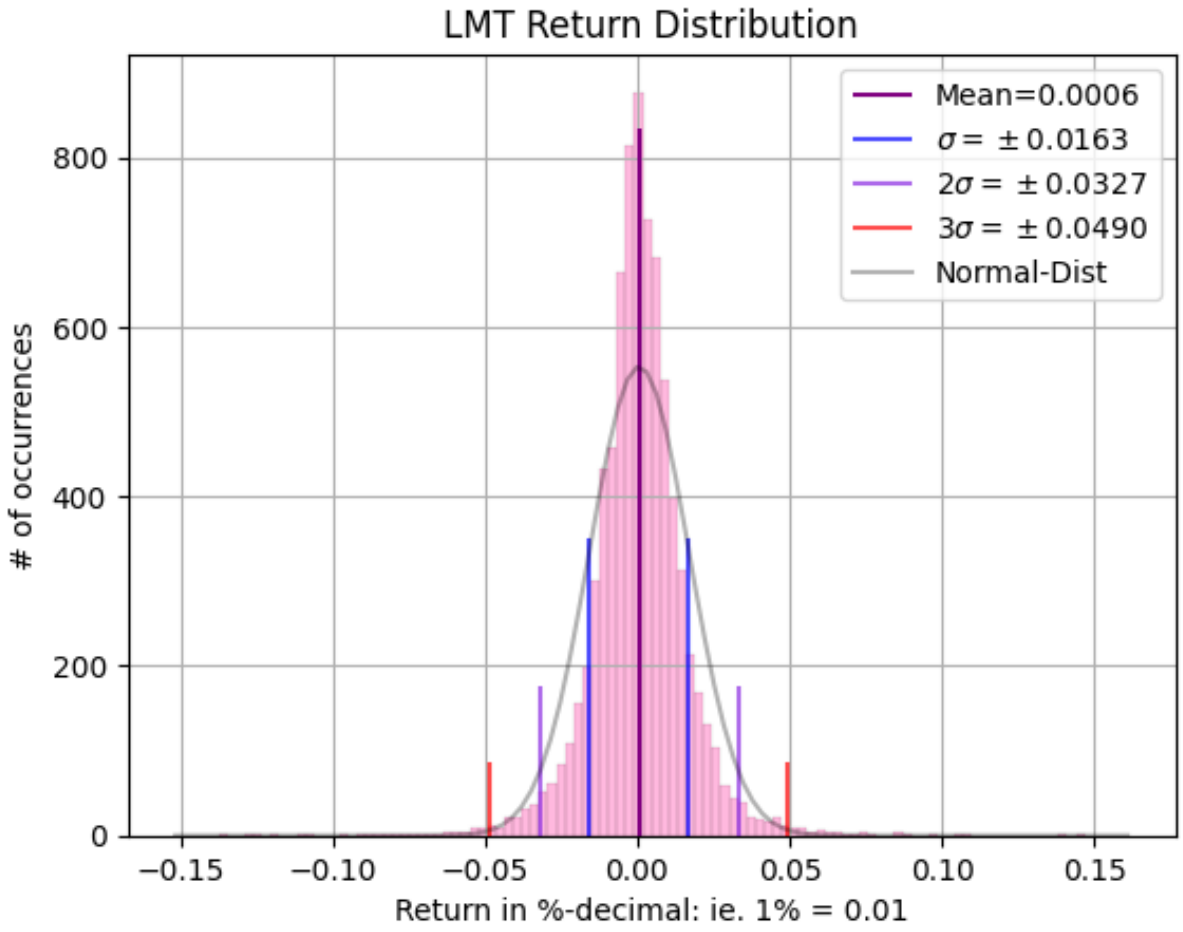
Out[10]:

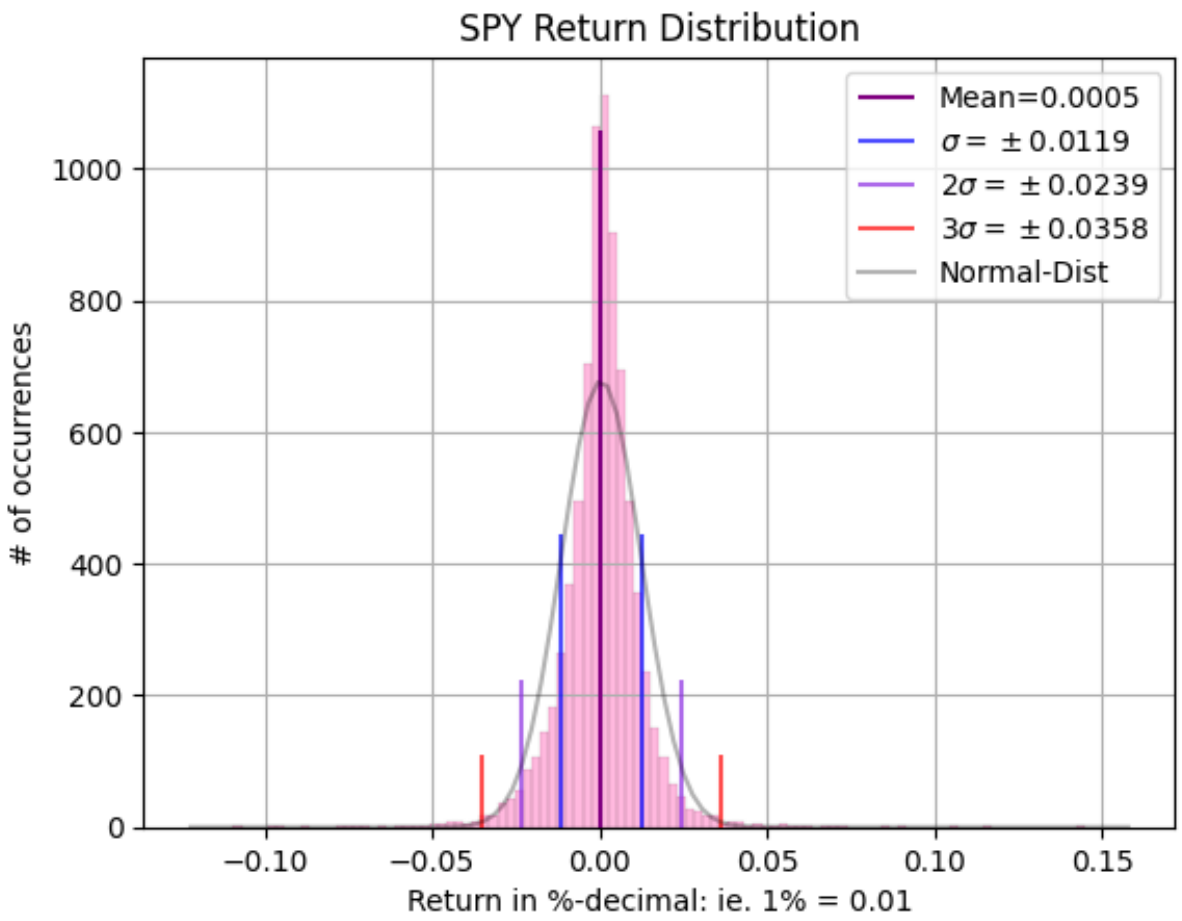
	AAPL	DX	LMT	SPY	TGT	TRMD
Mean	0.001234	0.000565	0.000637	0.000494	0.000719	0.001707
Median	0.000765	0.000000	0.000443	0.000726	0.000316	0.000000
Standard-Deviation	0.026248	0.030053	0.016346	0.011935	0.021066	0.037529
Skewness	-0.421460	1.564136	-0.086732	-0.012480	0.042343	2.983190
Kurtosis	27.746263	57.305824	8.586040	11.436700	11.011960	51.412531
Min	-0.518692	-0.470588	-0.137558	-0.109424	-0.249303	-0.306400
Max	0.332280	0.600000	0.146920	0.145198	0.204256	0.644737

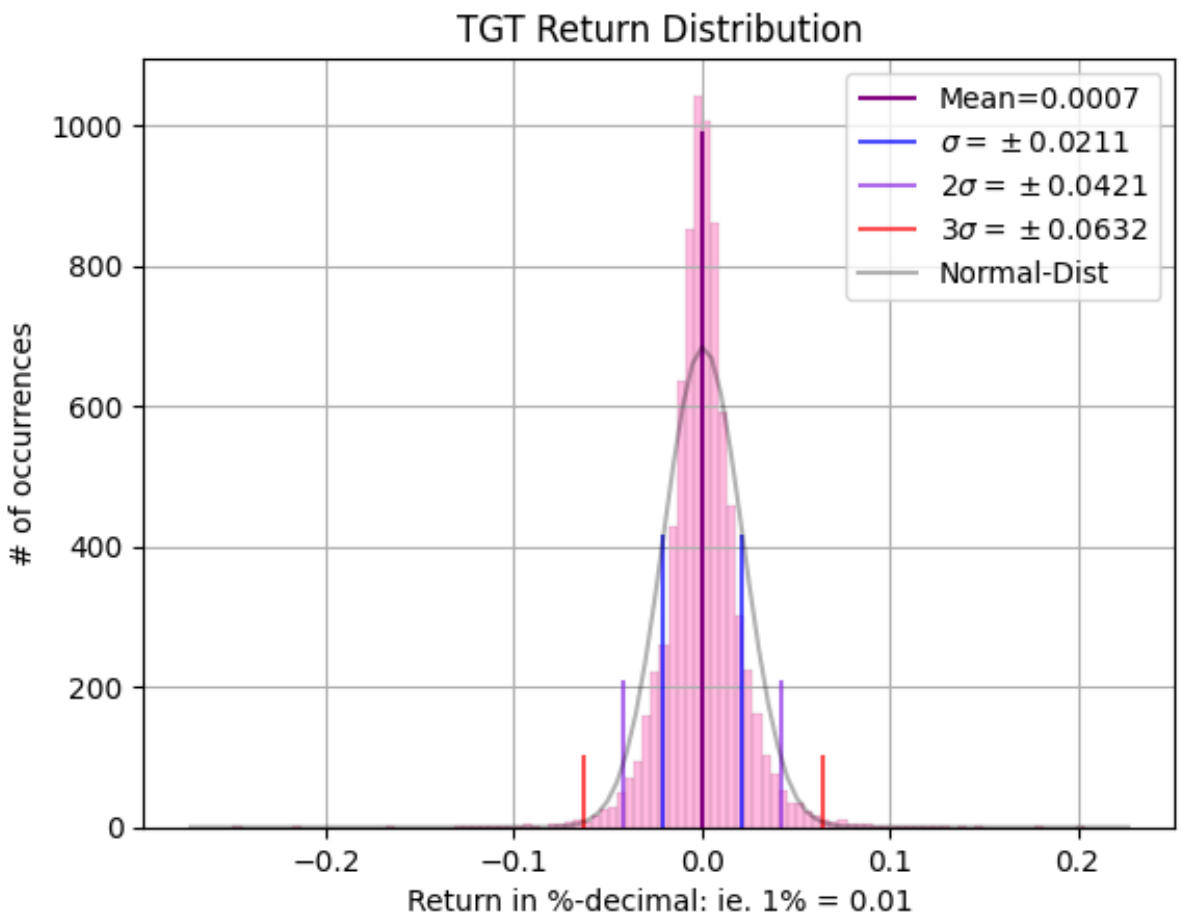
In [11]:
```
return_histograms(daily_returns)
```

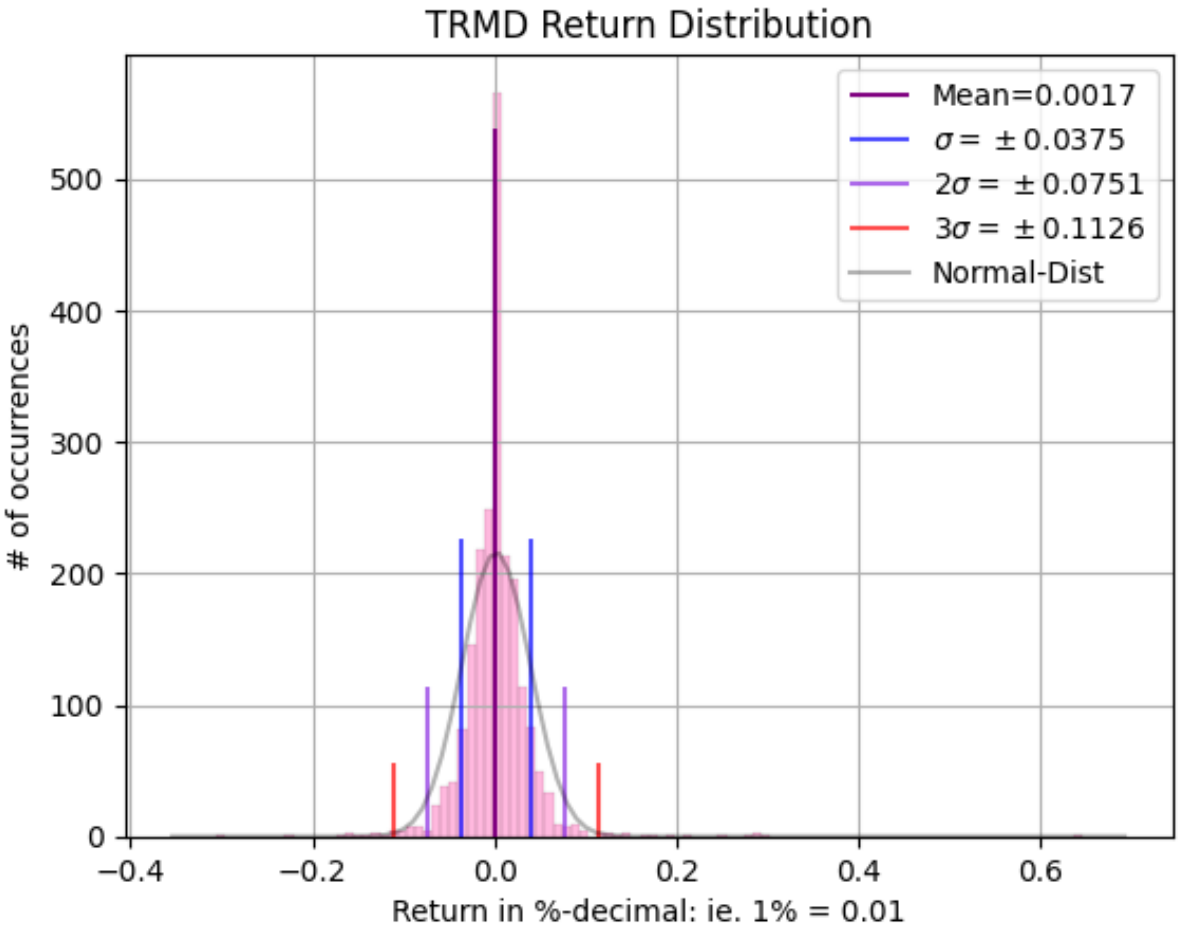



DX Return Distribution
Mean=0.0006
$\sigma = \pm 0.0301$
$2\sigma = \pm 0.0601$
$3\sigma = \pm 0.0902$
Normal-Dist
of occurrences
2500
2000
1500
1000
500
0
−0.4
−0.2
0.0
0.2
0.4
0.6
Return in %-decimal: ie. 1% = 0.01


LMT Return Distribution
Mean=0.0006
$\sigma = \pm 0.0163$
$2\sigma = \pm 0.0327$
$3\sigma = \pm 0.0490$
Normal-Dist
of occurrences
0
200
400
600
800
−0.15
−0.10
−0.05
0.00
0.05
0.10
0.15
Return in %-decimal: ie. 1% = 0.01


SPY Return Distribution
Mean=0.0005
$\sigma = \pm 0.0119$
$2\sigma = \pm 0.0239$
$3\sigma = \pm 0.0358$
Normal-Dist
of occurrences
0
200
400
600
800
1000
−0.10
−0.05
0.00
0.05
0.10
0.15
Return in %-decimal: ie. 1% = 0.01


TGT Return Distribution
Mean=0.0007
$\sigma = \pm 0.0211$
$2\sigma = \pm 0.0421$
$3\sigma = \pm 0.0632$
Normal-Dist
of occurrences
0
200
400
600
800
1000
−0.2
−0.1
0.0
0.1
0.2
Return in %-decimal: ie. 1% = 0.01



Reflect Task 2

Discussion

We are using real-world data here, so the distributions look a little 'jagged'. Notice the distribution of stock-returns *almost* follows a standard-normal distribution, *ie.* a bell-curve. Please, read the italicized *almost* as allllllmmmmmmmooooooosssssst. As asset-returns in financial markets are **NOT** normally distributed!

This bears repeating:

Financial asset returns do **NOT** follow a normal-distribution

Assuming returns follow a standard normal distribution allows a 'starting point' for analysis and discussion. It allows us to say things like *"the average daily return of this stock is 0.05%"* or *volatility*, taken from standard deviation, as the *average*

price movement of a stock.

Stock returns exhibit a higher-degree of *fat-tails events*, also known as *black-swan* events. This is evidenced by the extremely high-kurtosis, values between 8-50, where kurtosis of a Standard Normal Distribution = 0

Now that we have framed our normal assumption, we can look at the following.

Let's focus our attention on **AAPL** Apple

Apple is one of the highest growth stocks of the last 2-3 decades, returning +100,000% over this timeframe (1995-2026), yet it's **average daily return** is only ~0.12% with about ~49% of its trading days returning negative. In other words, the expected probability that Apple's stock return will be positive on any given day is just 51%. If you were a short-term trader on a daily time horizon on Apple, your odds of profit are *slightly* better than a 50/50 coin-flip. Apple's growth comes from the slight skew of positive returns and the kurtosis in the positive fat-tail. Those positive gains compounded over decades led to Apple's extraordinary performance, whilst the negative returns are given enough time to recover. The daily average movement is miniscule by comparison to decades of consistent price-perfromance from a stock like Apple.

Prompts

- Highest Average Daily Return :

  TRMD, Torm. Had an average daily return of ~0.16% The industrial shipping company beat out Apple suprisingly

- Highest Volatility :

  Again Torm with Volatility at ~3.7%

- Is Highest-Return ?= Highest-Vol. :

  Yes.

- Risk-Return

  This follows the notion that high-reward $\implies$ high-risk

Back to Top

Monthly Returns

In [12]:
```
# Here We just pull a snapshot of the price for each month. Should def
monthly_ticks = data = yf_tickers.history(start='1995-01-01', end='202
monthly_ticks = monthly_ticks['Close']
monthly_ticks
```

```
[**********************100%**************************]  6 of 6 completed
```

Out[12]:

Ticker	AAPL	DX	LMT	SPY	TGT	TRMD
Date						
1995-01-01	0.298493	6.290259	10.108562	27.134573	3.056603	NaN
1995-02-01	0.292827	7.759948	10.980509	28.241911	3.158041	NaN
1995-03-01	0.261320	7.341212	12.191541	29.028393	3.202834	NaN
1995-04-01	0.283560	7.762429	13.315583	29.888281	3.006857	NaN
1995-05-01	0.308971	8.905732	13.800618	31.074053	3.194707	NaN
...	...	...	...	...	...	...
2026-04-01	270.866638	12.742678	511.437775	715.037720	127.556534	29.624537
2026-05-01	311.791107	12.410752	523.760437	752.667114	126.102097	25.368906
2026-06-01	289.110657	12.593642	506.328247	744.920227	129.615143	24.269960
2026-07-01	308.643829	12.371807	579.157776	745.179626	143.389420	28.228031
2026-08-01	316.850006	12.710091	557.779968	765.149963	160.880005	30.556309

380 rows × 6 columns

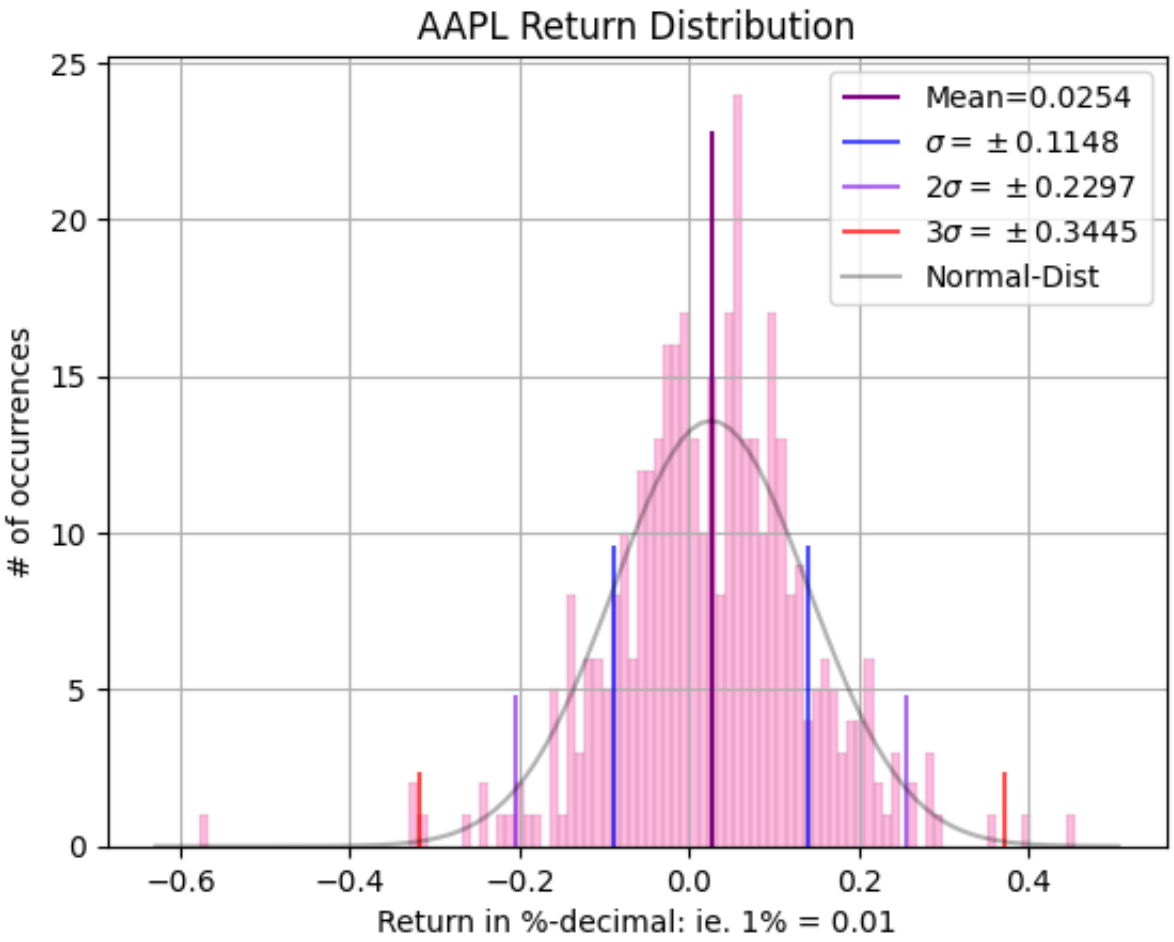
Monthly-Tick Returns

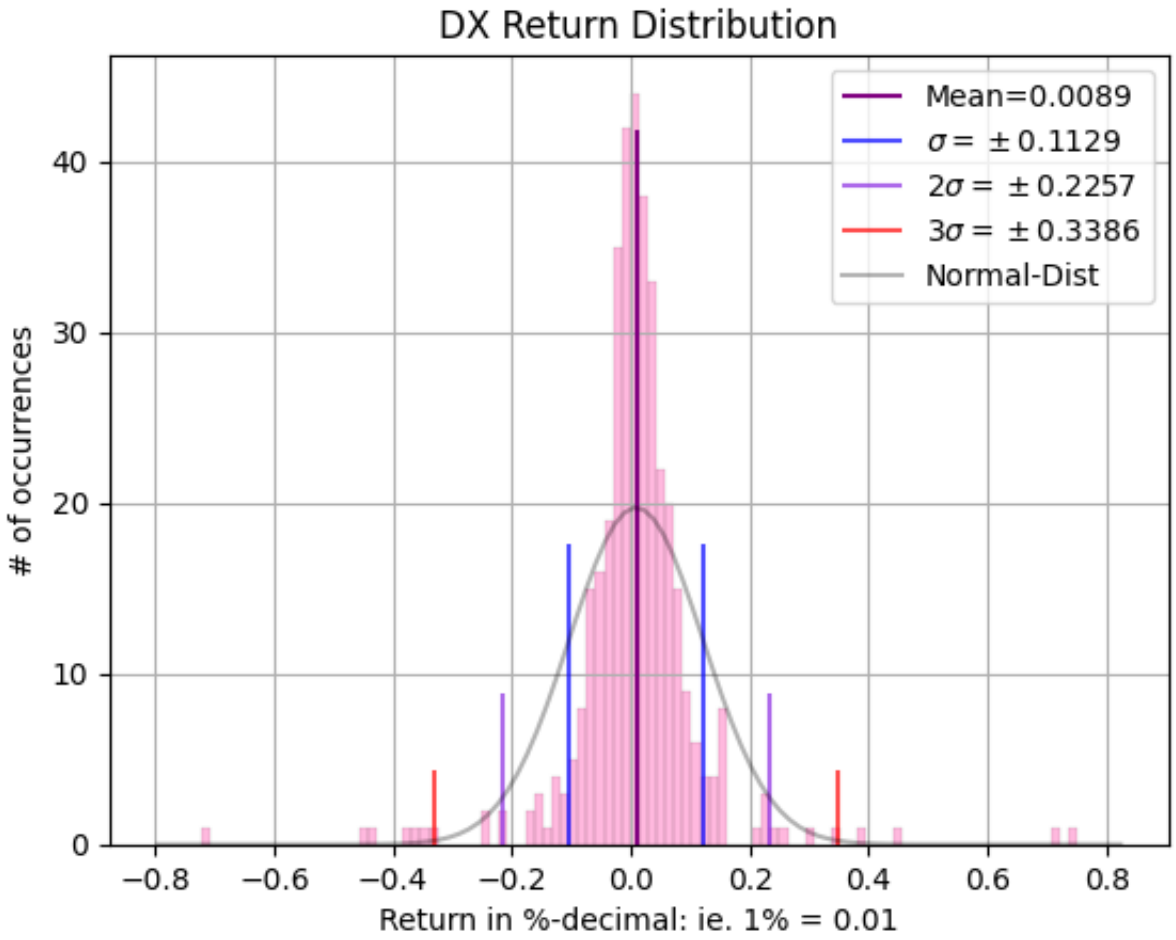
In [13]:
```
monthly_tick_return = monthly_ticks.pct_change()
summary_month_tick = summary_statistics_table(monthly_tick_return)
summary_month_tick
```

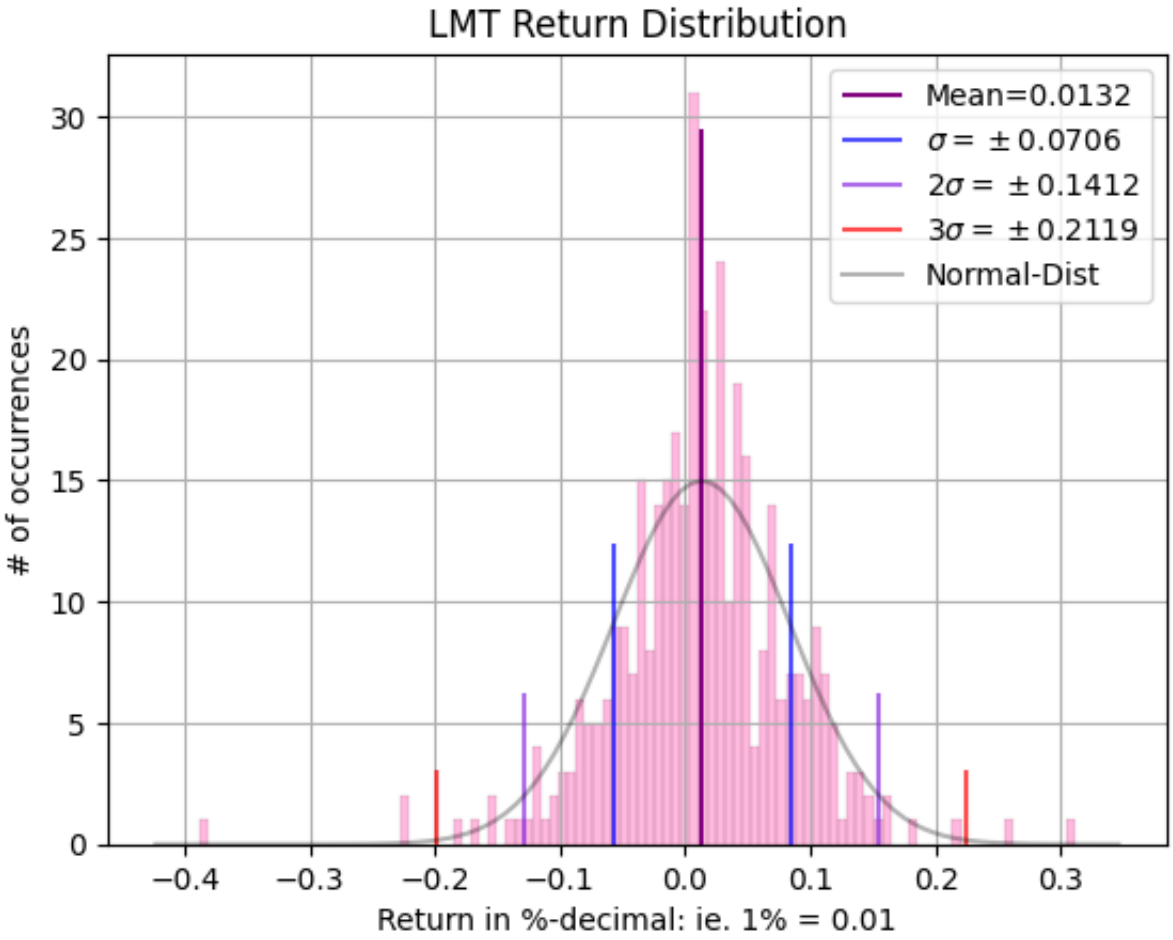
Out[13]:

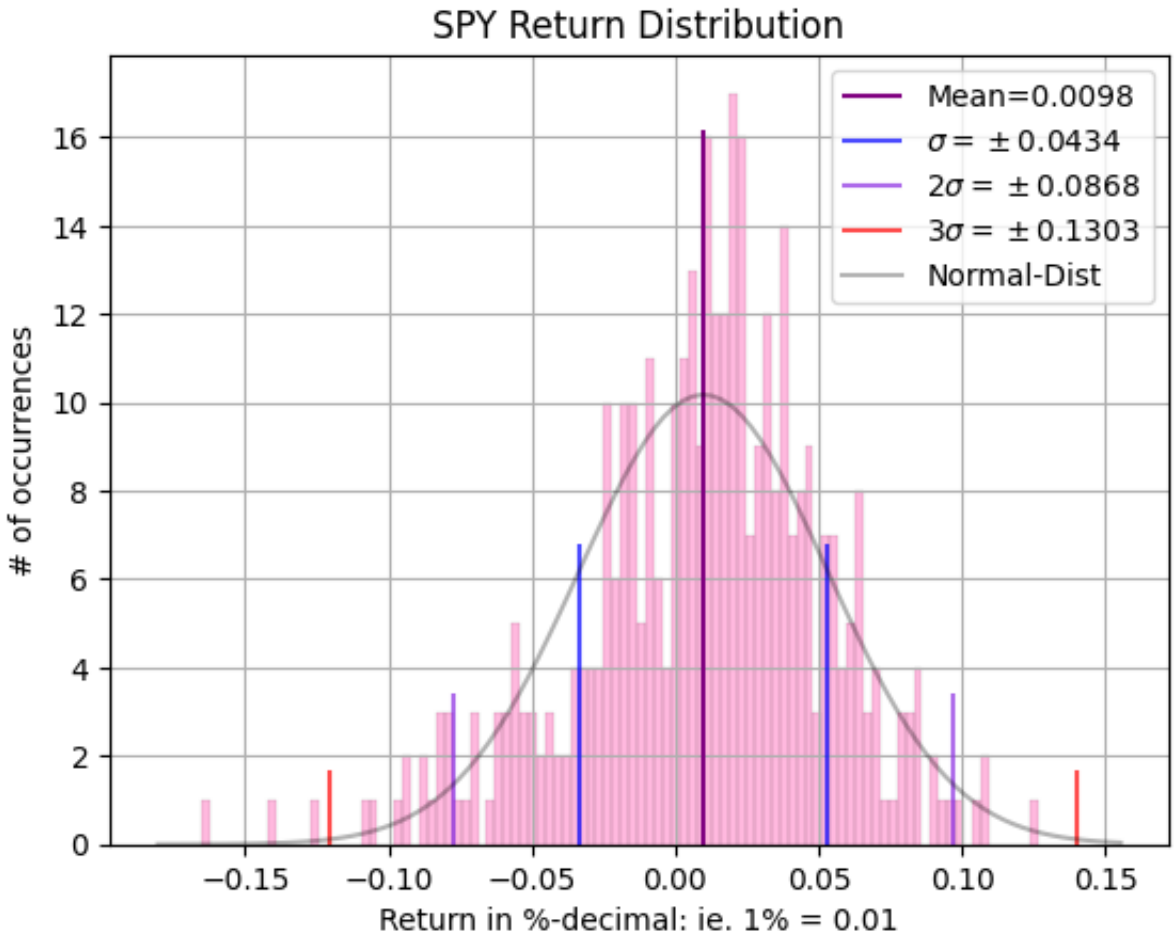
	AAPL	DX	LMT	SPY	TGT	TRMD
Mean	0.025378	0.008867	0.013182	0.009799	0.013834	0.028870
Median	0.027662	0.007103	0.011636	0.014113	0.012291	0.024620
Standard-Deviation	0.114828	0.112861	0.070618	0.043421	0.081976	0.120308
Skewness	-0.287919	0.357445	-0.395238	-0.566226	0.103426	0.460779
Kurtosis	2.493012	15.226930	3.767853	0.915856	0.617801	0.461175
Min	-0.577437	-0.722222	-0.388145	-0.165187	-0.289100	-0.239544
Max	0.453783	0.750000	0.311266	0.126983	0.266949	0.371545

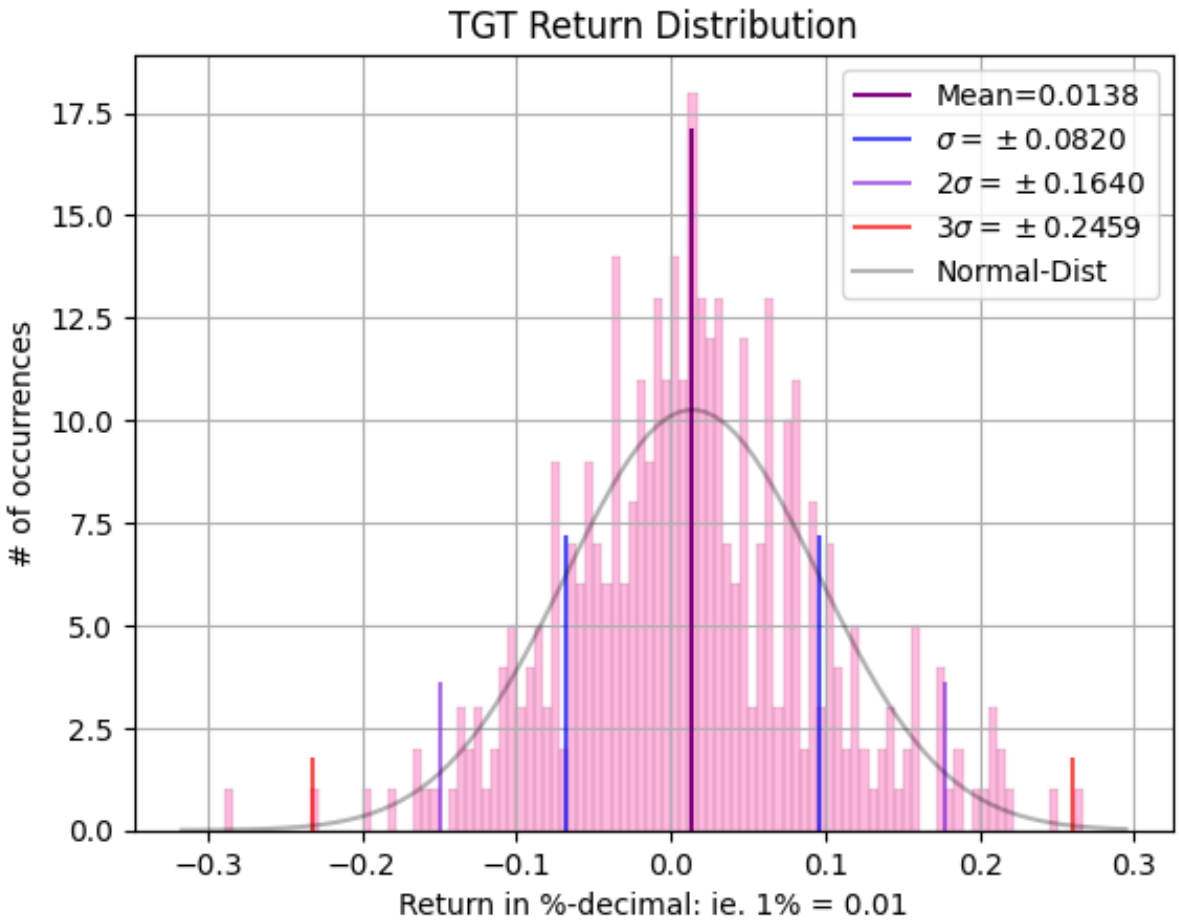
In [14]:
```
return_histograms(monthly_tick_return)
```

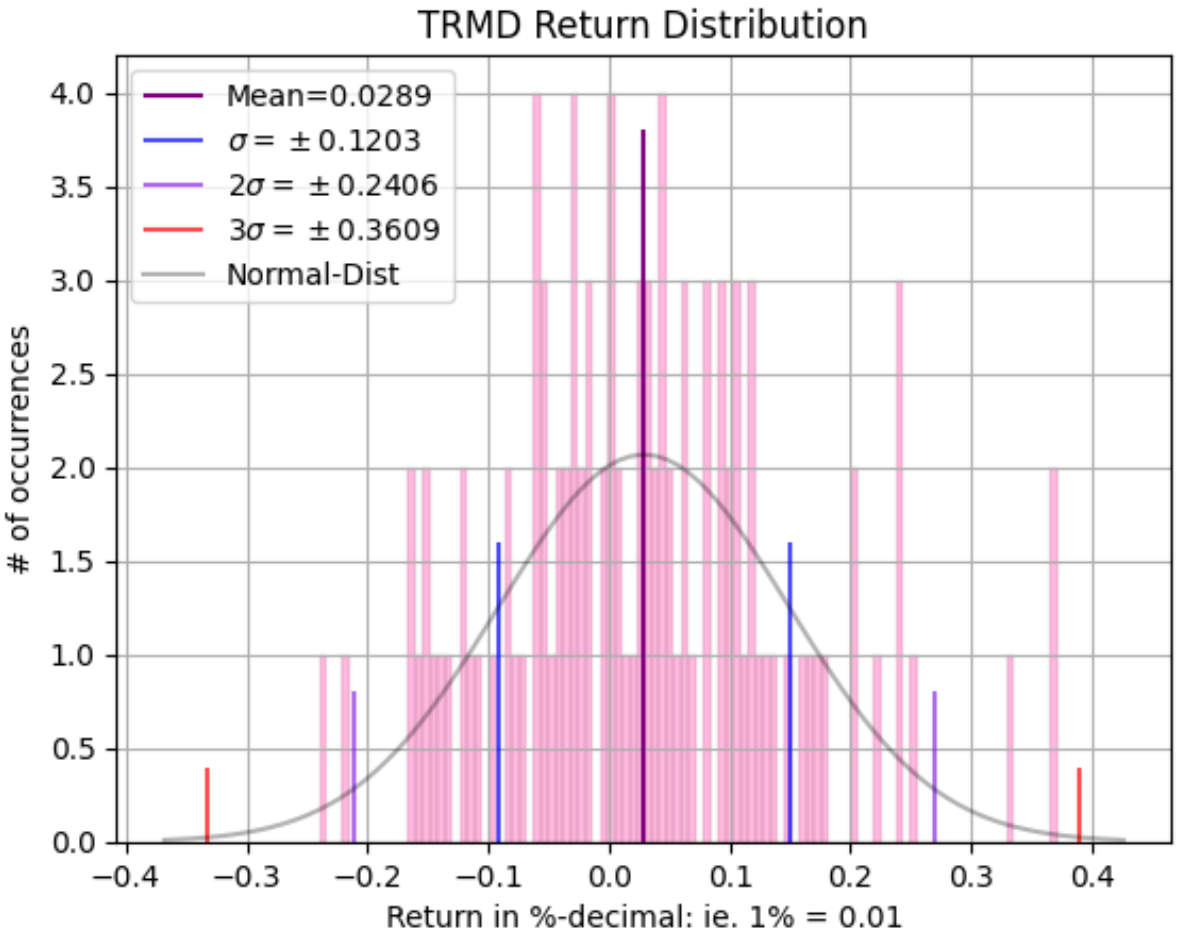



DX Return Distribution
Mean=0.0089
$\sigma = \pm 0.1129$
$2\sigma = \pm 0.2257$
$3\sigma = \pm 0.3386$
Normal-Dist
of occurrences
0
10
20
30
40
−0.8
−0.6
−0.4
−0.2
0.0
0.2
0.4
0.6
0.8
Return in %-decimal: ie. 1% = 0.01


LMT Return Distribution
Mean=0.0132
$\sigma = \pm 0.0706$
$2\sigma = \pm 0.1412$
$3\sigma = \pm 0.2119$
Normal-Dist
of occurrences
0
5
10
15
20
25
30
−0.4
−0.3
−0.2
−0.1
0.0
0.1
0.2
0.3
Return in %-decimal: ie. 1% = 0.01


SPY Return Distribution
Mean=0.0098
$\sigma = \pm 0.0434$
$2\sigma = \pm 0.0868$
$3\sigma = \pm 0.1303$
Normal-Dist
of occurrences
0
2
4
6
8
10
12
14
16
−0.15
−0.10
−0.05
0.00
0.05
0.10
0.15
Return in %-decimal: ie. 1% = 0.01


TGT Return Distribution
Mean=0.0138
$\sigma = \pm 0.0820$
$2\sigma = \pm 0.1640$
$3\sigma = \pm 0.2459$
Normal-Dist
of occurrences
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
−0.3
−0.2
−0.1
0.0
0.1
0.2
0.3
Return in %-decimal: ie. 1% = 0.01



Compare to Daily

Rescaled to monthly figure by just a simple times 21 (avg. trading days/month) with volatility scaled by $\sqrt{21}$

ignore the higher moments with simple rescaling

In [15]:
```
daily_returns.skew(), daily_returns.kurt()
```

Out[15]:
```
(Ticker
 AAPL   -0.421460
 DX      1.564136
 LMT    -0.086732
 SPY    -0.012480
 TGT     0.042343
 TRMD    2.983190
 dtype: float64,
 Ticker
 AAPL    27.746263
 DX      57.305824
 LMT      8.586040
 SPY     11.436700
 TGT     11.011960
 TRMD    51.412531
 dtype: float64)
```

In [16]:
```
daily_returns.std() * np.sqrt(21)
```

Out[16]:
```
Ticker
AAPL    0.120281
DX      0.137720
LMT     0.074907
SPY     0.054694
TGT     0.096538
TRMD    0.171981
dtype: float64
```

In [17]:
```
daily_to_month_summary = summary_statistics_table(daily_returns *21 )
daily_to_month_summary.loc[['Mean']]
```

Out[17]:

	AAPL	DX	LMT	SPY	TGT	TRMD
Mean	0.025919	0.011856	0.013384	0.010384	0.015104	0.035841

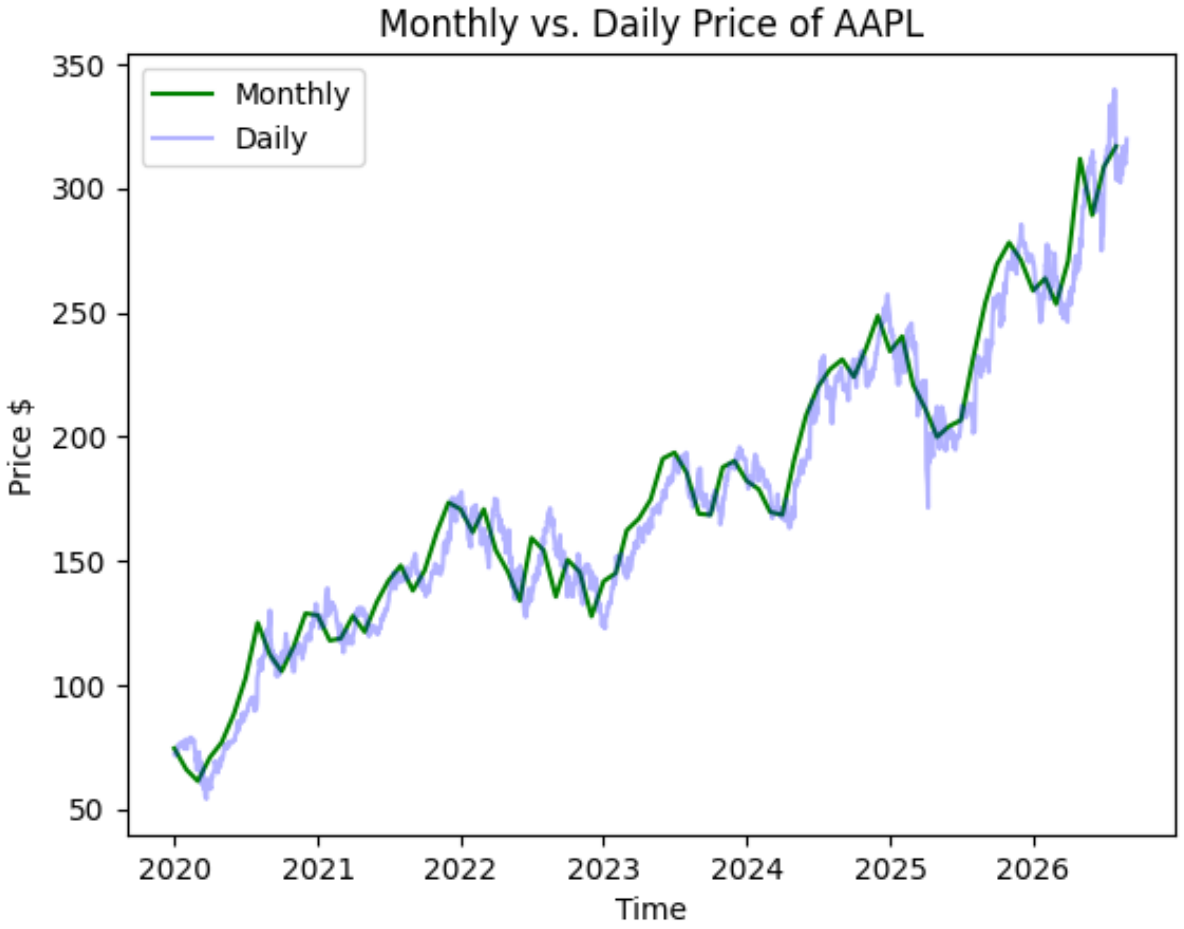
Reflect Task 3

Back to Top

Daily Volatility (Standard-Deviation) rescaled to monthly by $\sqrt{21}$ is higher then the monthly sample σ volatility. This makes sense as the daily figure captures more *'noise'* of the stock moving back and forth. By *'zooming out'* to monthly, we *smooth-out* the return profile, which has the effect of producing a lower volatility measure. Since the asset price can go up and down multiple times over the course of the month, some of those little up & down 'wiggles' are 'ignored' by the larger time sampling size.

Perhaps best illustrated with the following visualization, of daily return and monthly return plotted together for the same security (see below)

In [18]:
```
plt.plot(monthly_ticks.loc['2020':'2026']['AAPL'],color='green',label=
plt.plot(today.loc['2020':'2026']['AAPL'],color='blue', alpha=0.3, lab
plt.legend()
plt.title('Monthly vs. Daily Price of AAPL')
plt.xlabel('Time')
plt.ylabel('Price $')
plt.show()
```



In [31]:
```
for ticker in daily_returns.columns:
    stock_return = daily_returns[ticker]
    three_sigma = stock_return.std() * 3
    three_sigma_moves = stock_return.where(stock_return.abs() > three_
    print(three_sigma_moves)
```

```
Date
1995-01-06    0.080386
1995-03-14   -0.081968
1995-09-15   -0.103124
1995-12-18   -0.085108
1996-01-05    0.085151
                ...
```

```
2020-07-31    0.104689
2020-09-03   -0.080061
2022-11-10    0.088975
2025-04-03   -0.092456
2025-04-09    0.153289
Name: AAPL, Length: 114, dtype: float64
Date
1995-01-05    0.119048
1995-01-13    0.103448
1998-09-09   -0.134147
1998-10-05   -0.176923
1998-10-06   -0.121495
                ...
2020-03-30   -0.096991
2020-04-01   -0.109195
2020-04-06    0.112798
2020-04-08    0.173756
2022-04-27    0.095598
Name: DX, Length: 152, dtype: float64
Date
1995-04-24    0.053572
1995-06-19    0.049383
1997-04-04    0.055205
1997-10-27   -0.052191
1998-01-07    0.051768
                ...
2025-01-28   -0.091803
2025-03-21   -0.057934
2025-04-04   -0.049760
2025-07-22   -0.108115
2026-07-23    0.105432
Name: LMT, Length: 131, dtype: float64
Date
1997-10-27   -0.072474
1997-10-28    0.057706
1998-08-04   -0.038743
1998-08-27   -0.047073
1998-08-31   -0.071342
                ...
2022-11-10    0.054954
2025-04-03   -0.049281
2025-04-04   -0.058543
2025-04-09    0.105019
2025-04-10   -0.043819
Name: SPY, Length: 116, dtype: float64
Date
1995-11-02    0.076501
1997-10-27   -0.125261
1997-10-28    0.147971
1997-12-01    0.077140
1997-12-30    0.102822
                ...
```

```
2024-11-20   -0.214101
2025-04-03   -0.108632
2025-04-09    0.100608
2025-08-20   -0.063307
2026-03-03    0.067421
Name: TGT, Length: 119, dtype: float64
Date
2018-09-17   -0.151572
2018-11-07    0.174888
2018-11-20   -0.155715
2018-11-29    0.141397
2019-03-19    0.285476
2019-06-07    0.295838
2019-06-11    0.189344
2019-06-12   -0.160000
2019-08-20    0.248169
2019-10-09    0.126517
2020-01-27   -0.131873
2020-02-11    0.119722
2020-03-13    0.279859
2020-03-16   -0.166942
2020-03-18    0.216551
2020-03-19   -0.139909
2020-03-23   -0.125643
2020-04-21    0.644737
2020-04-22   -0.306400
2020-06-19    0.291156
2020-06-22   -0.227608
2021-10-18    0.143920
2021-11-08    0.123750
2022-05-11    0.160532
2022-06-21    0.136399
2022-07-06   -0.113459
2023-03-30   -0.127389
2025-03-06    0.117579
Name: TRMD, dtype: float64
```

In [33]:
```
for ticker in daily_returns.columns:
    stock_return = daily_returns[ticker]
    four5_sigma = stock_return.std() * 4.5
    four5_sigma_moves = stock_return.where(stock_return.abs() > three_
    print(four5_sigma_moves)
```

```
Date
1996-07-18    0.237035
1997-01-06   -0.178160
1997-07-11    0.146225
1997-08-06    0.332280
1998-01-02    0.238088
1998-01-06    0.192916
1998-10-09    0.139957
1999-04-27    0.117558
```

```
1999-09-21   -0.124110
1999-10-14    0.142996
2000-03-01    0.136858
2000-09-29   -0.518692
2000-12-06   -0.158088
2001-04-18    0.117155
2001-04-19    0.128566
2001-07-18   -0.171712
2002-06-19   -0.150372
2002-07-17   -0.124862
2003-05-05    0.113492
2004-07-15    0.113253
2004-10-14    0.131573
2006-07-20    0.118299
2008-09-29   -0.179195
2008-10-13    0.139050
2008-11-24    0.125575
2013-01-24   -0.123558
2020-03-13    0.119808
2020-03-16   -0.128647
2025-04-09    0.153289
Name: AAPL, dtype: float64
Date
1995-01-05    0.119048
1998-09-09   -0.134147
1998-10-05   -0.176923
1998-10-06   -0.121495
1998-10-07   -0.159575
                ...
2020-03-24   -0.214351
2020-03-25    0.146370
2020-03-26    0.347293
2020-04-06    0.112798
2020-04-08    0.173756
Name: DX, Length: 97, dtype: float64
Date
1999-06-09   -0.137558
1999-10-29   -0.128065
2000-04-24    0.140496
2001-09-17    0.146920
2020-03-12   -0.127616
2020-03-16   -0.123711
2021-10-26   -0.118034
Name: LMT, dtype: float64
Date
2008-10-13    0.145198
2008-10-28    0.116855
Name: SPY, dtype: float64
Date
1997-10-27   -0.125261
1997-10-28    0.147971
1998-08-31   -0.163584
```

```
1998-09-01    0.149660
2001-09-24    0.130060
2002-11-14    0.115218
2008-10-22   -0.113793
2008-10-28    0.178036
2008-12-01   -0.125000
2017-02-28   -0.121656
2019-08-21    0.204256
2019-11-20    0.140550
2020-03-12   -0.122751
2020-08-19    0.126516
2022-05-18   -0.249303
2022-11-16   -0.131356
2023-11-15    0.177543
2024-03-05    0.120207
2024-11-20   -0.214101
Name: TGT, dtype: float64
Date
2018-09-17   -0.151572
2018-11-07    0.174888
2018-11-20   -0.155715
2018-11-29    0.141397
2019-03-19    0.285476
2019-06-07    0.295838
2019-06-11    0.189344
2019-06-12   -0.160000
2019-08-20    0.248169
2019-10-09    0.126517
2020-01-27   -0.131873
2020-02-11    0.119722
2020-03-13    0.279859
2020-03-16   -0.166942
2020-03-18    0.216551
2020-03-19   -0.139909
2020-03-23   -0.125643
2020-04-21    0.644737
2020-04-22   -0.306400
2020-06-19    0.291156
2020-06-22   -0.227608
2021-10-18    0.143920
2021-11-08    0.123750
2022-05-11    0.160532
2022-06-21    0.136399
2022-07-06   -0.113459
2023-03-30   -0.127389
2025-03-06    0.117579
Name: TRMD, dtype: float64
```

In [ ]: